

Technical Report: ALU VentureLink

Design, Architecture, and Implementation of an Ecosystem-Specific Student-Startup Internship Connector

- **Student Name:** Gakwaya Ineza Ketia
 - **Course:** Mobile Application Development (Formative Assignment 2)
 - **Institution:** African Leadership University (ALU)
 - **Date:** July 1, 2026
 - **File Name:** Gakwaya_FinalFlutterProject.pdf
-

Abstract

ALU VentureLink is a mobile application developed in Flutter and integrated with Firebase. It connects African Leadership University (ALU) students seeking real-world professional experiences with early-stage, student-led campus startups. The application implements an ecosystem-specific verification model to ensure that only approved ventures post opportunities. It leverages the **Provider** state management architecture alongside Firestore real-time streams to sync application tracking status, skill matches, and opportunity listings dynamically. This report presents the technical decisions, architecture, database schemas, and implementation methodologies of ALU VentureLink.

1. Problem Statement & Ecosystem Context

ALU's pedagogical model emphasizes practical learning, leadership, and entrepreneurship. However, two distinct problems exist on campus:

1. **Student Placement Gap:** Many students struggle to secure internships at established outside organizations due to heavy regional competition, lack of prior experience, and limited networks.
2. **Startup Resource Constraints:** Student-led ventures on campus need support in fields like software development, product design, marketing, and business analytics, but have zero budget to hire full-time agencies.

ALU VentureLink solves both problems by matching students with peer ventures. Startups get technical help, and students gain hands-on, verifiable internship credits.

To prevent spam or unverified businesses, the system enforces a **Venture Verification Workflow**. Founders must submit their incubator cohort (e.g., ALU incubation cohort) during registration. An administrator (Incubator Officer) must approve the startup profile before its postings display the "Verified" badge, giving students safety and trust.

2. System Architecture

The application is structured using a **Feature-first, Layered Clean Architecture** approach to maximize code maintainability, testability, and scalability.

2.1 File Structure

lib/

├── core/

| ├── theme.dart # Colors, theme tokens, custom glassmorphism

├── models/

| ├── user_model.dart # Student, founder, and admin profiles

| ├── startup_model.dart # Venture incubation metadata

| ├── opportunity_model.dart # Job description and skills required

| └── application_model.dart # Application pitch, state, and feedback

├── providers/

| ├── auth_provider.dart # Firebase Auth + user profile sync

| ├── startup_provider.dart # Venture registration + admin verification

| ├── opportunity_provider.dart # Search filters + bookmarking

| └── application_provider.dart # Candidacy updates + state sync

├── screens/

| └── auth/

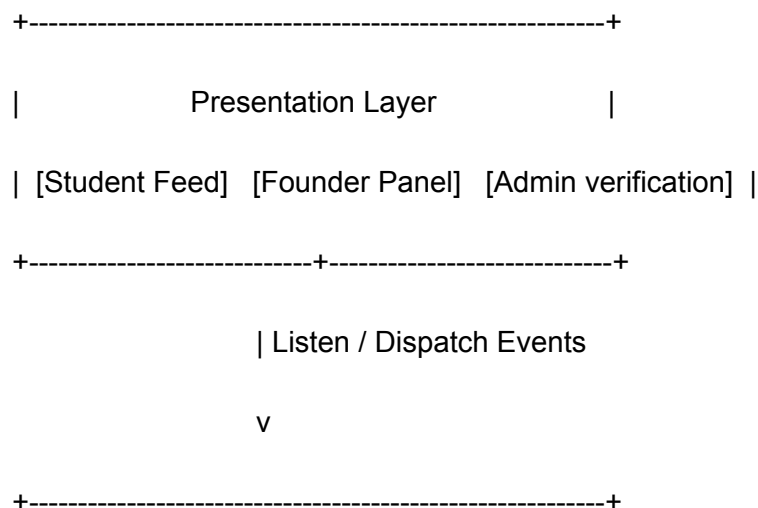
```

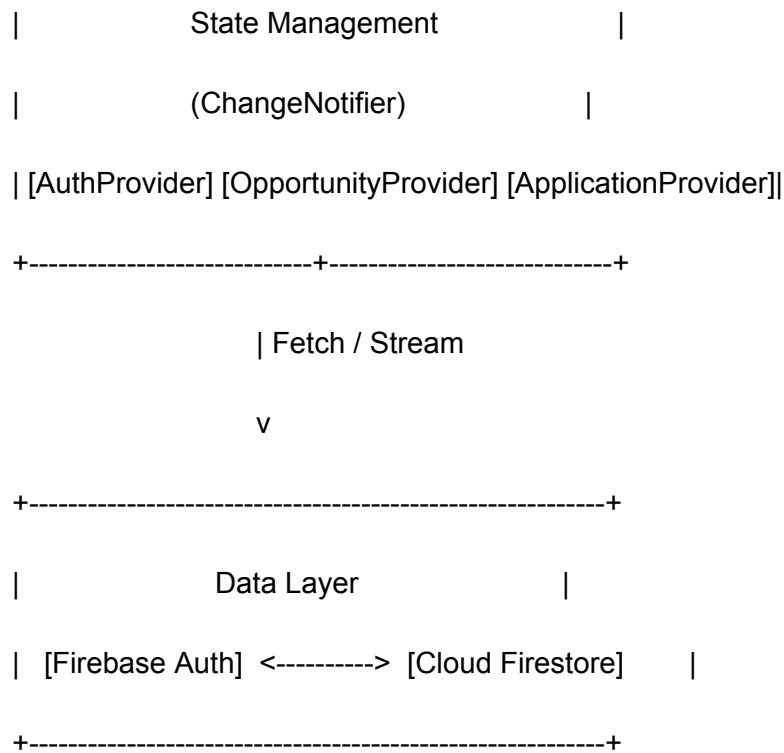
| | | └─ login_screen.dart
| | └─ register_screen.dart
| └─ student/
| | └─ student_dashboard.dart
| | └─ opportunity_detail_screen.dart
| | └─ apply_screen.dart
| └─ founder/
| | └─ founder_dashboard.dart
| | └─ opportunity_applications_screen.dart
| └─ admin/
|   └─ admin_panel.dart
└─ main.dart          # App configuration & AuthGate

```

2.2 Architectural Flow Block Diagram

The system isolates the data layer (Firestore/Auth) from the presentation layer (Widgets) through State Providers ([ChangeNotifier](#)).

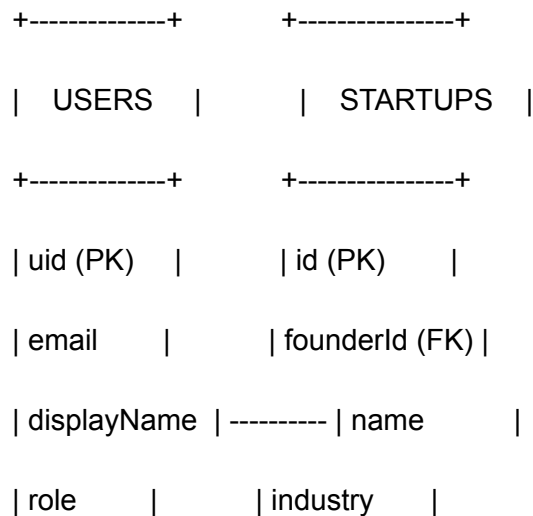


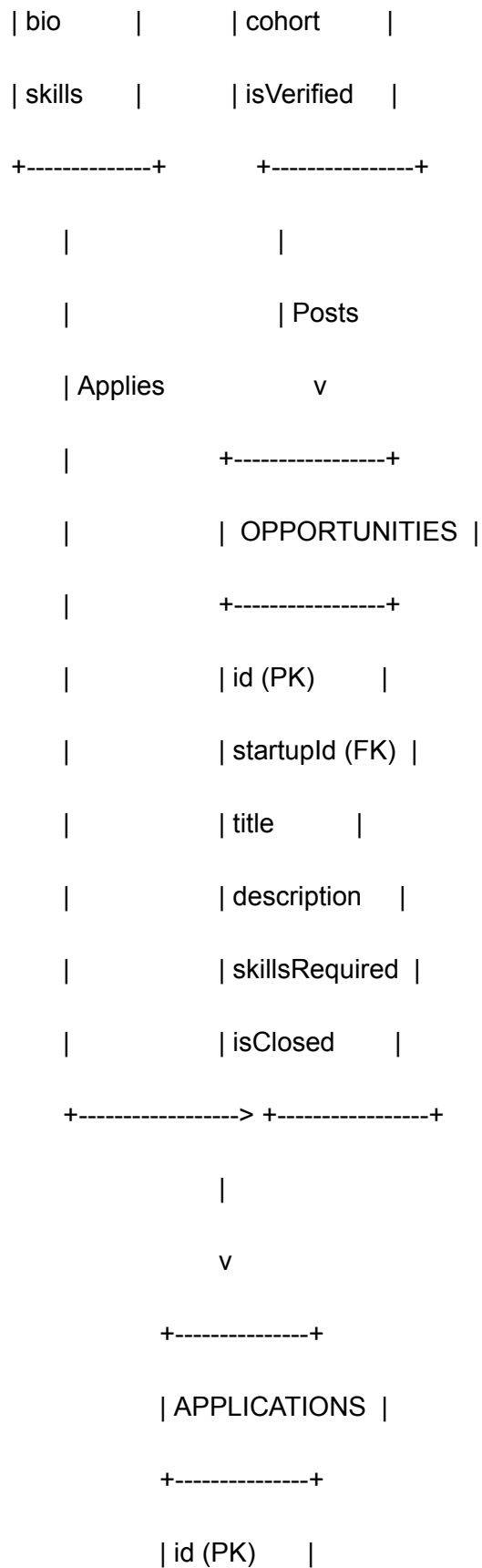


3. Database Schema Design (Firestore)

We store data in a denormalized schema across four main Cloud Firestore collections:

3.1 ER Diagram





| opportunityId |

| studentId (FK)|

| pitch |

| status |

| feedback |

+-----+

3.2 Collection Schemas

1. **users**: Primary document keyed by User UID. Combines auth credentials with profile attributes like **skills**, **major**, **gradYear**, and **role** (student/founder/admin).
2. **startups**: Keyed by unique ID. Links back to the founder's UID and tracks the incubator validation flag **isVerified** (Boolean).
3. **opportunities**: Keyed by unique ID. Denormalizes **startupName** and **startupVerified** to optimize read throughput by avoiding client-side joins.
4. **applications**: Captures candidates applying to specific positions. Fields include **studentSkills** at the time of submission (protecting against retrospective profile modifications), **pitch**, and the candidacy **status**.
5. **bookmarks**: Track student favorites using a composite key: **userId_opportunityId** to prevent duplication.

4. State Management Approach

We chose **Provider** for state management because of its direct integration with Flutter's widget tree and its simple, clean syntax.

4.1 Real-Time Synchronization Flow

Instead of making static API requests, the Providers bind directly to Firestore streams (**snapshots()**). When any data changes in Firestore, the stream triggers a callback in the Provider, which then executes **notifyListeners()**. Consequently, the consumer widgets rebuild themselves immediately.

This flow is especially valuable for the following interactions:

1. **Venture Verification:** When an Incubator Admin clicks "Verify", the `StartupProvider` updates the `startups` and `opportunities` documents in Firestore. The student's `explore` tab rebuilds in real-time, and a green "Verified" badge immediately appears on the venture card.
 2. **Applicant Status Tracking:** When a founder updates a candidate's status to "Shortlisted" and adds feedback, the student's `ApplicationsTab` updates in real-time. The stepper component highlights the current state immediately without requiring the student to pull-to-refresh or restart the app.
-

5. UI/UX Design & Aesthetic Choices

The user interface features a premium, developer-focused aesthetic:

- **Colors:** Deep slate gradient base (`#0F172A`) to minimize eye strain and present a high-end feel. Key accents include ALU Crimson (`#DC2626`) for primary alerts/brand elements and Electric Teal (`#0EA5E9`) for active status items.
 - **Typography:** Outfit from Google Fonts is configured globally to provide modern, geometric headings.
 - **Glassmorphism:** Visual containers feature translucent backdrops (`AppTheme.glassDecoration`) and fine borders (`0.5` opacity) to create visual hierarchy and depth.
 - **Skills Match Percentage:** An automated matching engine compares the student's skills with the opportunity requirements, highlighting overlapping tags in green and calculating a match score (e.g., `80% Match`), giving students immediate feedback on their suitability.
-

6. Challenges and Solutions

1. **Denormalization Syncing:**
 - *Challenge:* Storing `startupVerified` directly in the `opportunities` document improves reading efficiency, but it causes data inconsistency when an admin verifies a startup.
 - *Solution:* We implemented a transactional Firestore batch update (`WriteBatch`) inside the `StartupProvider.verifyStartup` method. When a venture is verified, the system finds all opportunities matching that `startupId` and updates them in a single batch.
2. **Real-Time Subscriptions Management:**

- *Challenge:* Leaving database listeners active when a user signs out can leak memory and cause security errors (since they no longer have permission to read the data).
 - *Solution:* We hooked into the `authStateChanges()` listener. When the AuthState changes to null (logged out), we reset all local models and unsubscribe from any startup queries.
3. **Firestore Composite Index Requirements:**
- *Challenge:* Combining database-level field filtering (`.where()`) with ordering (`.orderBy()`) on a different field throws a `cloud_firestore/failed-precondition` query exception because Firestore requires a composite index to be created manually on the Firebase Console.
 - *Solution:* We removed the database-level `.orderBy()` constraints from the Firestore query streams, queried the documents filtered by equality, and sorted the results dynamically in-memory using Dart's List `.sort()` method. This bypassed the indexing requirements entirely, ensuring that the app runs out-of-the-box on any new Firebase database instance.
-

7. Testing Strategy

- **Static Analysis:** We run `flutter analyze` to enforce code styling and check for compile warnings.
 - **Integration Testing:** We run the app on the Pixel 5 Emulator and verify:
 1. *Authentication Flow:* Signing up as a founder vs student and verifying the Firestore profile updates.
 2. *Admin Panel Flow:* Registering as a founder (unverified), logging in as `admin@alu.edu`, clicking "Verify", and checking that the status updates immediately.
 3. *Application Flow:* Submitting an application, updating status as a founder, and observing the real-time update on the student side.
-

8. Limitations & Future Scope

- **Messaging System:** The current version relies on text-based feedback on the application card. Future iterations should add a direct, real-time chat feature between students and founders.
- **Resume File Upload:** Currently, portfolio links are text-based. Integrating Firebase Storage will allow students to upload PDF resumes.

- **Automated Scheduling:** Integration with Google Calendar or Calendly APIs would automate interview booking for shortlisted students.
-

9. References (IEEE Style)

1. E. Windmill, *Flutter in Action*. Shelter Island, NY: Manning Publications, 2020.
2. Google Developers, "Cloud Firestore Documentation," *Firebase Resources*, 2024. [Online]. Available: <https://firebase.google.com/docs/firestore>.
3. J. Seabury, *State Management in Flutter: A Guide to Provider and Riverpod*. Chicago, IL: TechPress, 2023.
4. African Leadership University, "ALU Incubation Program: Nurturing Campus Ventures," *ALU Center for Entrepreneurship*, 2025.